Com760_Group28 Code Submission

1. ***Exploratory_Data_Analysis***

```
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
import numpy as np

# Load dataset
df = pd.read_excel("oasis_longitudinal_demographics.xlsx")

# Style
sns.set_theme(style="whitegrid")

# ---------------------------------------------------------
# 1. Age vs Brain Volume
# ---------------------------------------------------------
plt.figure(figsize=(8,5))
sns.scatterplot(data=df, x='Age', y='nWBV', hue='Group')
plt.title('Brain Volume vs Age by Group')
plt.show()

# ---------------------------------------------------------
# 2. MMSE vs Group
# ---------------------------------------------------------
plt.figure(figsize=(8,5))
sns.boxplot(data=df, x='Group', y='MMSE')
plt.title('MMSE vs Dementia Group')
plt.show()

# ---------------------------------------------------------
# 3. Age Distribution
# ---------------------------------------------------------
plt.figure(figsize=(8,5))
sns.histplot(df['Age'], kde=True)
plt.title('Age Distribution')
plt.show()

# ---------------------------------------------------------
# 4. Correlation Heatmap
# ---------------------------------------------------------
plt.figure(figsize=(10,8))
sns.heatmap(df.select_dtypes(include=['number']).corr(), annot=True,
cmap='coolwarm')
```

```python
plt.title('Correlation Matrix')
plt.show()

# ---------------------------------------------------------
# 5. Longitudinal Analysis (Patient History)
# ---------------------------------------------------------
plt.figure(figsize=(10,6))

for subject in df['Subject ID'].unique()[:5]:
    subj_data = df[df['Subject ID'] == subject].sort_values('Visit')
    plt.plot(subj_data['Visit'], subj_data['nWBV'], marker='o', label=subject)

plt.title('Patient Brain Volume Change Over Time')
plt.xlabel('Visit Number')
plt.ylabel('nWBV')
plt.legend()
plt.show()

# ---------------------------------------------------------
# 6. Dementia Progression (Converted Patients)
# ---------------------------------------------------------
converters = df[df['Group'] == 'Converted']['Subject ID'].unique()[:5]

plt.figure(figsize=(10,6))

for subject in converters:
    subj_data = df[df['Subject ID'] == subject].sort_values('Visit')
    plt.plot(subj_data['Visit'], subj_data['nWBV'], marker='o', label=subject)

plt.title('Brain Volume Decline in Converted Patients')
plt.xlabel('Visit')
plt.ylabel('nWBV')
plt.legend()
plt.show()

# ---------------------------------------------------------
# 7. Regression Analysis (Normal vs Dementia)
# ---------------------------------------------------------
plt.figure(figsize=(8,5))

sns.regplot(
    data=df[df['Group']=='Nondemented'],
    x='Age', y='nWBV',
    label='Nondemented'
)

sns.regplot(
    data=df[df['Group']=='Demented'],
```

```
    x='Age', y='nWBV',
    color='red',
    label='Demented'
)

plt.title('Brain Atrophy Comparison')
plt.legend()
plt.show()

# ---------------------------------------------------------
# 8. Summary Statistics
# ---------------------------------------------------------
print("Summary Statistics:")
print(df.groupby('Group')[['Age','MMSE','nWBV']].mean())
```

2. ***Data_Cleaning and Feature_engineering***

```
import pandas as pd
from sklearn.model_selection import train_test_split

# Data Cleaning

def load_and_clean(path):
    df = pd.read_excel(path)

    df = df.drop(columns=["MRI ID"])
    df = df.drop(columns=["CDR"])

    df["M/F"] = df["M/F"].map({"M": 1, "F": 0})

    if "Hand" in df.columns and df["Hand"].nunique() == 1:
        df = df.drop(columns=["Hand"])

    return df

# Feature Engineering

def add_features(df):
    df["Brain_Ratio"] = df["nWBV"] / df["eTIV"]
    df["Age_nWBV"] = df["Age"] * df["nWBV"]

    df = df.sort_values(["Subject ID", "Visit"])
    df["nWBV_diff"] = df.groupby("Subject ID")["nWBV"].diff().fillna(0)
    df["MMSE_diff"] = df.groupby("Subject ID")["MMSE"].diff().fillna(0)
```

```
    return df

def split_data(df):
    X = df.drop(columns=["Group", "Subject ID"])
    y = df["Group"]

    print(f"\nData shape: {X.shape}")
    print(f"Classes: {", ".join(y.unique())}")

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42, stratify=y
    )

    return X,y,X_train, X_test, y_train, y_test
```

3. *Model Training*

```
import pandas as pd
import numpy as np
import joblib
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.impute import KNNImputer
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (confusion_matrix, accuracy_score,
precision_score,
                             recall_score, f1_score, classification_report)
from xgboost import XGBClassifier
import os

def train_and_save(X, X_train, X_test, y_train, y_test):

    os.makedirs("models", exist_ok=True)
    os.makedirs("outputs", exist_ok=True)

    imputer = KNNImputer(n_neighbors=5)
    scaler = StandardScaler()

    X_train_processed = imputer.fit_transform(X_train)
    X_train_processed = scaler.fit_transform(X_train_processed)

    X_test_processed = imputer.transform(X_test)
    X_test_processed = scaler.transform(X_test_processed)

    X_train_df = pd.DataFrame(X_train_processed, columns=X.columns)
    X_test_df = pd.DataFrame(X_test_processed, columns=X.columns)
```

```python
    # STAGE 1: DEMENTED vs REST (Converted + Nondemented)
    print("\nSTAGE 1: DEMENTED vs REST")
    print("_"*30)

    # Binary: Demented (1) vs Rest (0)
    y_train_stage1 = np.where(y_train == "Demented", 1, 0)
    y_test_stage1 = np.where(y_test == "Demented", 1, 0)

    model_stage1 = XGBClassifier(
        n_estimators=300,
        max_depth=4,
        learning_rate=0.05,
        subsample=0.8,
        colsample_bytree=0.8,
        random_state=42,
        eval_metric="logloss"
    )

    model_stage1.fit(X_train_df, y_train_stage1)
    y_pred_stage1_train = model_stage1.predict(X_train_df)
    y_pred_stage1_test = model_stage1.predict(X_test_df)

    # Evaluate Stage 1
    evaluate_model(y_test_stage1, y_pred_stage1_test, "Stage 1 (Demented vs
Rest)",
                   ["Rest", "Demented"], "binary")

    # STAGE 2: CONVERTED vs NONDEMENTED
    # (Only on non-demented samples from training)
    print("\nSTAGE 2: CONVERTED vs NONDEMENTED")
    print("_"*30)

    train_mask = (y_train != "Demented").values
    X_train_stage2 = X_train_df[train_mask].reset_index(drop=True)
    y_train_stage2_raw = y_train[train_mask].reset_index(drop=True)

    # Binary labels: Converted (1) vs Nondemented (0)
    y_train_stage2 = np.where(y_train_stage2_raw == "Converted", 1, 0)

    test_mask = (y_test != "Demented").values
    X_test_stage2 = X_test_df[test_mask].reset_index(drop=True)
    y_test_stage2_raw = y_test[test_mask].reset_index(drop=True)
    y_test_stage2_true = np.where(y_test_stage2_raw == "Converted", 1, 0)

    model_stage2 = XGBClassifier(
        n_estimators=400,
        max_depth=5,
```

```
        learning_rate=0.03,
        subsample=0.8,
        colsample_bytree=0.8,
        random_state=42,
        eval_metric="logloss"
    )

    model_stage2.fit(X_train_stage2, y_train_stage2)

    # Predict on test samples that Stage 1 classified as non-demented
    indices_for_stage2 = np.where(y_pred_stage1_test == 0)[0]
    X_test_for_stage2 =
X_test_df.iloc[indices_for_stage2].reset_index(drop=True)
    y_pred_stage2 = model_stage2.predict(X_test_for_stage2)

    y_test_stage2_true_for_eval = np.where(y_test.iloc[indices_for_stage2] ==
"Converted", 1, 0)

    # Evaluate Stage 2
    if len(y_test_stage2_true_for_eval) > 0:
        evaluate_model(y_test_stage2_true_for_eval, y_pred_stage2,
                       "Stage 2 (Converted vs Nondemented)",
                       ["Nondemented", "Converted"], "binary")

    # FINAL PREDICTION COMBINATION
    print("\nFINAL HIERARCHICAL PREDICTIONS")
    print("_"*30)

    final_preds = np.empty(len(X_test_df), dtype=object)

    final_preds[y_pred_stage1_test == 1] = "Demented"

    for i, original_idx in enumerate(indices_for_stage2):
        if y_pred_stage2[i] == 1:
            final_preds[original_idx] = "Converted"
        else:
            final_preds[original_idx] = "Nondemented"

    # Evaluate final predictions
    print(f"\nFinal Classification Report (3-class):\n")
    print(classification_report(y_test, final_preds, zero_division=0))

    # Confusion matrix for final predictions
    cm_final = confusion_matrix(y_test, final_preds, labels=["Nondemented",
"Converted", "Demented"])

    plt.figure(figsize=(10, 8))
    sns.heatmap(cm_final, annot=True, fmt='d', cmap='Blues',
```

```
                xticklabels=["Nondemented", "Converted", "Demented"],
                yticklabels=["Nondemented", "Converted", "Demented"],
                cbar_kws={'label': 'Count'})
    plt.title('Confusion Matrix - Final Hierarchical Predictions',
fontsize=14, fontweight='bold')
    plt.ylabel('True Label', fontsize=12)
    plt.xlabel('Predicted Label', fontsize=12)
    plt.tight_layout()

    filename = "outputs/confusion_matrix_final_hierarchical.png"
    plt.savefig(filename, dpi=300, bbox_inches='tight')
    print(f"\nFinal confusion matrix saved to outputs/")
    plt.close()

    # Saving Models and preprocessing
    joblib.dump(model_stage1, "models/model_stage1.pkl")
    joblib.dump(model_stage2, "models/model_stage2.pkl")
    joblib.dump(imputer, "models/imputer.pkl")
    joblib.dump(scaler, "models/scaler.pkl")
    joblib.dump(X.columns.tolist(), "models/feature_columns.pkl")

    print("\nTraining complete and models saved.")
    print("_"*30)

    return model_stage1, model_stage2, imputer, scaler


def evaluate_model(y_true, y_pred, model_name, class_labels, model_type):

    accuracy = accuracy_score(y_true, y_pred)
    precision = precision_score(y_true, y_pred, average='weighted',
zero_division=0)
    recall = recall_score(y_true, y_pred, average='weighted', zero_division=0)
    f1 = f1_score(y_true, y_pred, average='weighted', zero_division=0)

    print(f"\n{model_name} - Performance Metrics:")
    print(f"  Accuracy:  {accuracy:.4f}")
    print(f"  Precision: {precision:.4f}")
    print(f"  Recall:    {recall:.4f}")
    print(f"  F1-Score:  {f1:.4f}")

    print(f"\nDetailed Classification Report:")
    print(classification_report(y_true, y_pred, target_names=class_labels,
                                zero_division=0))

    # confusion matrix visualization
    cm = confusion_matrix(y_true, y_pred)

    plt.figure(figsize=(10, 8))
```

```
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                xticklabels=class_labels, yticklabels=class_labels,
                cbar_kws={'label': 'Count'})
    plt.title(f'Confusion Matrix - {model_name}', fontsize=14,
fontweight='bold')
    plt.ylabel('True Label', fontsize=12)
    plt.xlabel('Predicted Label', fontsize=12)
    plt.tight_layout()

    filename = f"outputs/confusion_matrix_{model_name.replace(' ',
'_').lower()}.png"
    plt.savefig(filename, dpi=300, bbox_inches='tight')
    print(f"\nConfusion matrix saved to outputs/")
    plt.close()
```

4. ***Model_Evaluation and Inference***

```
import pandas as pd
import numpy as np
import joblib
import shap
import os
from pathlib import Path

# Global model cache
_models_cache = {}

PROJECT_ROOT = Path(__file__).parent.parent
MODELS_DIR = PROJECT_ROOT / "models"

def _load_models():
    """Lazy load models on first use."""
    global _models_cache

    if _models_cache:
        return _models_cache

    required_files = [
        MODELS_DIR / "model_stage1.pkl",
        MODELS_DIR / "model_stage2.pkl",
        MODELS_DIR / "imputer.pkl",
        MODELS_DIR / "scaler.pkl",
        MODELS_DIR / "feature_columns.pkl"
    ]

    missing = [str(f) for f in required_files if not f.exists()]
```

```
    if missing:
        raise FileNotFoundError(
            f"Models not found: {missing}\n"
            "Please run training first (e.g. `python main.py` with
MODE='train')."
        )

    model_stage1 = joblib.load(str(MODELS_DIR / "model_stage1.pkl"))
    model_stage2 = joblib.load(str(MODELS_DIR / "model_stage2.pkl"))
    imputer = joblib.load(str(MODELS_DIR / "imputer.pkl"))
    scaler = joblib.load(str(MODELS_DIR / "scaler.pkl"))
    feature_columns = joblib.load(str(MODELS_DIR / "feature_columns.pkl"))

    _models_cache = {
        'model_stage1': model_stage1,
        'model_stage2': model_stage2,
        'imputer': imputer,
        'scaler': scaler,
        'feature_columns': feature_columns,
        'explainer_stage1': shap.Explainer(model_stage1),
        'explainer_stage2': shap.Explainer(model_stage2),
    }

    return _models_cache

# FEATURE MEANING MAP
feature_meaning = {
    "MMSE": "cognitive score",
    "nWBV": "brain volume",
    "eTIV": "intracranial volume",
    "Age": "age",
    "M/F": "gender",
    "Brain_Ratio": "brain-to-skull ratio",
    "Age_nWBV": "age * brain volume",
    "nWBV_diff": "brain volume change",
    "MMSE_diff": "cognitive decline"
}


# SHAP
def generate_text_explanation(shap_values, input_df, feature_names, class_idx,
top_n=5):

    if isinstance(shap_values, list):
        shap_vals = shap_values[class_idx][0]
    else:
        sv = shap_values.values

        if len(sv.shape) == 3:
```

```
            shap_vals = sv[0, :, class_idx]
        elif len(sv.shape) == 2:
            if sv.shape[1] == len(feature_names):
                shap_vals = sv[0]
            else:
                shap_vals = sv[0, class_idx]
        else:
            shap_vals = sv[0]

    feature_data = []
    for i, feature in enumerate(feature_names):
        feature_data.append({
            "feature": feature,
            "value": input_df.iloc[0, i],
            "impact": shap_vals[i]
        })

    # Sort by absolute importance
    feature_data = sorted(feature_data, key=lambda x: abs(x["impact"]),
reverse=True)

    # Get top features
    top_features = feature_data[:3]
    other_features = feature_data[3:top_n]

    def format_feature(f):
        name = feature_meaning.get(f["feature"], f["feature"])
        value = round(f['value'], 2)

        if f["impact"] > 0:
            phrase = "-> higher risk"
        else:
            phrase = "-> lower risk"

        return f"{name} ({value}) {phrase}"

    top_sentences = [format_feature(f) for f in top_features]
    other_sentences = [format_feature(f) for f in other_features]

    return top_sentences, other_sentences

def preprocess(df):
    models = _load_models()
    df = df.copy()
    # Ensure columns order matches training
    df = df[models["feature_columns"]]
    X = models['imputer'].transform(df)
    X = models['scaler'].transform(X)
```

```python
    return pd.DataFrame(X, columns=df.columns)

def predict_patient(df):
    models = _load_models()
    X = preprocess(df)
    feature_names = X.columns.tolist()

    # Stage 1: Demented vs Rest (binary)
    prob1 = models['model_stage1'].predict_proba(X)[0]
    pred1 = np.argmax(prob1)
    confidence1 = round(prob1[pred1] * 100, 2)

    shap_vals1 = models['explainer_stage1'](X)

    # Training encodes Demented=1, Rest=0
    if pred1 == 0:
        top, others = generate_text_explanation(
            shap_vals1, df, feature_names, class_idx=1
        )

        return {
            "prediction": "Nondemented",
            "confidence": confidence1,
            "top_signals": top,
            "other_factors": others
        }

    # Stage 2: Converted vs Nondemented (binary on non-demented patients)
    prob2 = models['model_stage2'].predict_proba(X)[0]
    pred2 = int(np.argmax(prob2))
    confidence2 = round(prob2[pred2] * 100, 2)

    final_label = "Converted" if pred2 == 1 else "Nondemented"

    shap_vals2 = models['explainer_stage2'](X)
    top, others = generate_text_explanation(shap_vals2, df, feature_names,
class_idx=1)

    return {
        "prediction": final_label,
        "confidence": confidence2,
        "top_signals": top,
        "other_factors": others,
    }
```

5. *Shap Analysis*

```python
#!/usr/bin/env python

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import shap
import joblib
import os
from pathlib import Path
from sklearn.preprocessing import LabelEncoder
from xgboost import XGBClassifier

from data_processing import load_and_clean, add_features
from shap_visualizations import (
    load_shap_data,
    prepare_shap_data,
    get_shap_explainer,
    get_feature_importance,
    analyze_feature_contribution
)

# Set style
sns.set_style("whitegrid")
plt.rcParams['figure.figsize'] = (14, 8)

# Create outputs directory
os.makedirs("outputs", exist_ok=True)

print("\nSHAP MODEL INTERPRETABILITY ANALYSIS - COMPLETE SUITE")

print("\n Loading data and models...")
# Resolve data path relative to project root so running from any CWD works.
PROJECT_ROOT = Path(__file__).parent.parent
data_path = PROJECT_ROOT / "data" / "oasis_longitudinal_demographics.xlsx"
df = load_and_clean(str(data_path))
df = add_features(df)

X = df.drop(columns=["Group", "Subject ID"])
y = df["Group"]

print(f"    Data shape: {X.shape}")
print(f"    Classes: {y.unique().tolist()}")
```

```python
# Load models
model2, imputer, scaler, le = load_shap_data(model_dir="models")
X_processed = prepare_shap_data(X, imputer, scaler)

print(f"    Model2 (Loaded): {type(model2).__name__}")
print(f"    Preprocessed shape: {X_processed.shape}")

expected_classes = sorted(y.unique().tolist())
loaded_n_classes = None
try:
    loaded_n_classes = int(getattr(model2, "n_classes_",
model2.predict_proba(X_processed.iloc[[0]]).shape[1]))
except Exception:
    loaded_n_classes = None

# If the loaded model isn't truly multiclass, train a multiclass model for
SHAP so the
# feature-importance plot matches the 3-class stacked bar style (as in the
reference image).
model_for_shap = model2
le_for_shap = le
if loaded_n_classes is not None and loaded_n_classes != len(expected_classes):
    print(f"    Note: loaded model predicts {loaded_n_classes} classes, but
data has {len(expected_classes)} classes.")
    print("    Training a multiclass XGBoost model for SHAP plots...")

    le_for_shap = LabelEncoder()
    y_enc = le_for_shap.fit_transform(y)

    model_for_shap = XGBClassifier(
        objective="multi:softprob",
        num_class=len(le_for_shap.classes_),
        eval_metric="mlogloss",
        n_estimators=400,
        learning_rate=0.05,
        max_depth=4,
        subsample=0.9,
        colsample_bytree=0.9,
        reg_lambda=1.0,
        random_state=42,
        n_jobs=-1,
    )
    model_for_shap.fit(X_processed, y_enc)

print(f"    SHAP model: {type(model_for_shap).__name__}
(classes={len(getattr(le_for_shap, 'classes_', []))})")


print("\n Computing SHAP values (this may take a moment)...")
```

```
explainer, predictions = get_shap_explainer(model_for_shap, X_processed)
shap_values = explainer(X_processed)

print(f"   - SHAP values computed: {shap_values.values.shape}")

# For multiclass models, SHAP may come back as 2D depending on explainer.
# To match the stacked-per-class bar plot (like the reference image),
# recompute per-class SHAP values when possible.
shap_values_for_bar = shap_values
shap_values_for_importance = shap_values
try:
    if hasattr(model2, "predict_proba") and getattr(model2, "n_classes_", 0) >
2 and shap_values.values.ndim == 2:
        tree_explainer = shap.TreeExplainer(model2)
        per_class = tree_explainer.shap_values(X_processed)  # list-of-arrays
or (n, f, c)
        shap_values_for_bar = per_class
        shap_values_for_importance = per_class
        if isinstance(per_class, list):
            print(f"   - Recomputed per-class SHAP values:
({per_class[0].shape[0]}, {per_class[0].shape[1]}, {len(per_class)})")
        else:
            print(f"   - Recomputed per-class SHAP values: {per_class.shape}")
except Exception as e:
    print(f"   - Note: per-class SHAP recomputation skipped
({type(e).__name__}: {e})")


print("\n Analyzing feature importance...")

importance_df = get_feature_importance(
    shap_values_for_importance,
    X_processed.columns.tolist(),
    top_n=15
)

print("\n   Top 15 Features by Importance:")
print("   " + "-" * 50)
for idx, row in importance_df.iterrows():
    print(f"   {idx+1:2d}. {row['feature']:15s} -> {row['importance']:.4f}")

# Save to CSV
importance_df.to_csv("outputs/shap_feature_importance.csv", index=False)
print(f"\n    Saved to: outputs/shap_feature_importance.csv")

print("\n Generating visualizations...")
print("   1. Creating feature importance bar plot...")

plt.figure(figsize=(12, 8))
```

```
shap.summary_plot(shap_values_for_bar, X_processed, plot_type="bar",
show=False)
plt.title("Feature Importance - Mean Absolute SHAP Values", fontsize=14,
fontweight='bold')
plt.tight_layout()
plt.savefig("outputs/01_shap_importance_bar.png", dpi=300,
bbox_inches='tight')
plt.close()
print("      - Saved: outputs/01_shap_importance_bar.png")

# Get top 4 features for later use
top_4_features = importance_df.head(4)['feature'].tolist()


print("   2. Creating SHAP values beeswarm plot...")

plt.figure(figsize=(14, 10))
# SHAP can be either 2D (n_samples, n_features) or 3D (n_samples, n_features,
n_classes)
sv = shap_values.values
if sv.ndim == 3:
    shap_values_summed = np.abs(sv).mean(axis=2)  # Average across classes
else:
    shap_values_summed = sv
shap.summary_plot(
    shap.Explanation(values=shap_values_summed,
base_values=np.mean(shap_values_summed),
                     data=X_processed.values,
feature_names=X_processed.columns.tolist()),
    X_processed,
    plot_type="dot",
    show=False
)
plt.title("SHAP Value Distribution by Feature (Mean Across Classes)",
fontsize=14, fontweight='bold')
plt.tight_layout()
plt.savefig("outputs/02_shap_values_distribution.png", dpi=300,
bbox_inches='tight')
plt.close()
print("      - Saved: outputs/02_shap_values_distribution.png")

print("   3. Creating dependence plots for top 4 features...")

# For multiclass, average SHAP values across classes; otherwise just use
absolute SHAP values
if sv.ndim == 3:
    shap_values_mean = np.abs(sv).mean(axis=2)
else:
    shap_values_mean = np.abs(sv)
```

```
shap_values_mean_df = pd.DataFrame(shap_values_mean,
columns=X_processed.columns)

fig, axes = plt.subplots(2, 2, figsize=(14, 10))
axes = axes.flatten()

for idx, feature in enumerate(top_4_features):
    axes[idx].scatter(X_processed[feature], shap_values_mean_df[feature],
alpha=0.6, s=50)
    axes[idx].set_xlabel(feature, fontweight='bold')
    axes[idx].set_ylabel('Mean |SHAP value|', fontweight='bold')
    axes[idx].set_title(f"Dependence: {feature}", fontweight='bold')
    axes[idx].grid(True, alpha=0.3)

plt.suptitle("Feature Dependence Plots - Top 4 Features", fontsize=14,
fontweight='bold')
plt.tight_layout()
plt.savefig("outputs/03_shap_dependence_plots.png", dpi=300,
bbox_inches='tight')
plt.close()
print("      - Saved: outputs/03_shap_dependence_plots.png")


print("   4. Creating sample predictions with SHAP details...")

# Get predictions for different classes
class_labels = list(getattr(le_for_shap, "classes_", []))
fig, axes = plt.subplots(1, len(class_labels), figsize=(max(16, 5 *
len(class_labels)), 5))
if len(class_labels) == 1:
    axes = [axes]

class_indices = []
for class_label in class_labels:
    # Find first prediction of each class
    preds = model_for_shap.predict(X_processed)
    pred_classes = le_for_shap.inverse_transform(preds)
    idx = np.where(pred_classes == class_label)[0]
    if len(idx) > 0:
        class_indices.append(idx[0])

for ax, sample_idx in zip(axes, class_indices):
    pred_class = le_for_shap.inverse_transform(
        [np.argmax(model_for_shap.predict_proba(X_processed.iloc[[sample_idx]]
))]
    )[0]

    # Get top 10 SHAP values for this sample
```

```
    sv_abs = np.abs(shap_values.values[sample_idx]).mean(axis=-1) if
len(shap_values.values.shape) == 3 else np.abs(shap_values.values[sample_idx])
    top_indices = np.argsort(sv_abs)[-10:][::-1]

    top_features = X_processed.columns[top_indices].tolist()
    top_values = sv_abs[top_indices]

    ax.barh(range(len(top_features)), top_values, color='steelblue')
    ax.set_yticks(range(len(top_features)))
    ax.set_yticklabels(top_features)
    ax.set_xlabel('Mean |SHAP value|')
    ax.set_title(f'Sample {sample_idx}: {pred_class}', fontweight='bold')
    ax.invert_yaxis()

plt.suptitle("Top 10 SHAP Values for Sample Predictions", fontsize=14,
fontweight='bold')
plt.tight_layout()
plt.savefig("outputs/04_shap_sample_explanations.png", dpi=300,
bbox_inches='tight')
plt.close()
print("       - Saved: outputs/04_shap_sample_explanations.png")


print("\n Generating detailed feature analysis...")

top_10_features = importance_df.head(10)['feature'].tolist()
analysis_data = []

for feature in top_10_features:
    stats = analyze_feature_contribution(explainer, shap_values, X_processed,
feature)
    analysis_data.append(stats)

analysis_df = pd.DataFrame(analysis_data)
analysis_df.to_csv("outputs/shap_feature_contribution_stats.csv", index=False)

print("\n   Feature Contribution Statistics (Top 10):")

print(f"   {'Feature':<15} {'Mean':<10} {'Median':<10} {'Std':<10}
{'Range':<25} {'Dir':<15}")

for _, row in analysis_df.iterrows():
    range_str = f"[{row['max_negative']:.3f}, {row['max_positive']:.3f}]"
    direction = f"{row['positive_count']}/{row['negative_count']}"
    print(f"   {row['feature']:<15} {row['mean_contribution']:<10.4f} "
          f"{row['median_contribution']:<10.4f}
{row['std_contribution']:<10.4f} "
          f"{range_str:<25} {direction:<15}")
```

```
print("\n Saved: outputs/shap_feature_contribution_stats.csv")

print(" ANALYSIS SUMMARY")

print(f"""
KEY FINDINGS:

1. TOP 5 MOST IMPORTANT FEATURES:
   {chr(10).join([f'   {i+1}. {row["feature"]:<15} (importance:
{row["importance"]:.4f})'
                  for i, (_, row) in
enumerate(importance_df.head(5).iterrows())])}

2. FEATURE IMPACT INTERPRETATION:
   - Positive SHAP values  - Increase prediction likelihood
   - Negative SHAP values  - Decrease prediction likelihood
   - Larger magnitude      - Stronger influence on prediction

3. MODEL INSIGHTS:
   - Total samples analyzed:    {X_processed.shape[0]}
   - Total features:            {X_processed.shape[1]}
   - Number of classes:         {len(class_labels)}
   - Classes:                   {', '.join(class_labels)}

""")

print("SHAP ANALYSIS COMPLETE!")
```

6. ***Shap_Analysis(logic)***

```
import shap
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
import joblib
import os
from pathlib import Path

# Get project root (parent of src/)
PROJECT_ROOT = Path(__file__).parent.parent

def load_shap_data(model_dir="models"):
```

```python
    # Convert to absolute path if relative
    if not os.path.isabs(model_dir):
        model_dir = PROJECT_ROOT / model_dir
    else:
        model_dir = Path(model_dir)

    # Allow either legacy names (model2.pkl) or stage naming
(model_stage2.pkl)
    model2_path = model_dir / "model2.pkl"
    if not model2_path.exists():
        stage2_path = model_dir / "model_stage2.pkl"
        if stage2_path.exists():
            model2_path = stage2_path

    required_files = [model2_path, model_dir / "imputer.pkl", model_dir /
"scaler.pkl", model_dir / "le.pkl"]
    missing = [str(f) for f in required_files if not f.exists()]
    if missing:
        raise FileNotFoundError(f"Missing model files: {missing}")

    model2 = joblib.load(str(model2_path))
    imputer = joblib.load(str(model_dir / "imputer.pkl"))
    scaler = joblib.load(str(model_dir / "scaler.pkl"))
    le = joblib.load(str(model_dir / "le.pkl"))

    return model2, imputer, scaler, le


def prepare_shap_data(X_df, imputer, scaler):
    """Preprocess data for SHAP analysis."""
    X_processed = imputer.transform(X_df)
    X_processed = scaler.transform(X_processed)
    return pd.DataFrame(X_processed, columns=X_df.columns)


def get_shap_explainer(model, X_processed):
    """Create SHAP explainer."""
    return shap.Explainer(model), model.predict(X_processed)


def plot_force_plot(explainer, shap_values, X_processed, sample_idx=0):
    """Plot SHAP force plot for a single sample."""
    return shap.force_plot(
        explainer.expected_value,
        shap_values.values[sample_idx],
        X_processed.iloc[sample_idx],
        matplotlib=True
    )
```

```python
def plot_summary_plot(explainer, shap_values, X_processed, plot_type="bar"):

    if plot_type == "bar":
        shap.summary_plot(shap_values, X_processed, plot_type="bar")
    else:
        shap.summary_plot(shap_values, X_processed)
    plt.tight_layout()


def plot_dependence_plot(explainer, shap_values, X_processed, feature_name):
    shap.dependence_plot(feature_name, shap_values.values, X_processed)
    plt.tight_layout()


def plot_waterfall_plot(explainer, shap_values, X_processed, sample_idx=0):
    return shap.plots.waterfall(
        shap.Explanation(
            values=shap_values.values[sample_idx],
            base_values=explainer.expected_value,
            data=X_processed.iloc[sample_idx],
            feature_names=X_processed.columns.tolist()
        )
    )


def get_feature_importance(shap_values, feature_names, top_n=10):

    # Accept SHAP as Explanation, ndarray, or list-of-arrays (multiclass
TreeExplainer)
    if hasattr(shap_values, "values"):
        sv = shap_values.values
    else:
        sv = shap_values

    if isinstance(sv, list):
        # List of (n_samples, n_features) per class -> (n_samples, n_features,
n_classes)
        sv = np.stack(sv, axis=-1)

    if len(sv.shape) == 3:
        # Multiclass: (n_samples, n_features, n_classes)
        # Take mean absolute value across all samples and classes
        mean_shap = np.abs(sv).mean(axis=(0, 2))
    else:
        # Binary: (n_samples, n_features)
        mean_shap = np.abs(sv).mean(axis=0)

    importance_df = pd.DataFrame({
        'feature': feature_names,
```

```
        'importance': mean_shap
    }).sort_values('importance', ascending=False)

    return importance_df.head(top_n)


def analyze_feature_contribution(explainer, shap_values, X_processed,
feature_name):

    feature_idx = X_processed.columns.tolist().index(feature_name)
    # Accept SHAP as Explanation, ndarray, or list-of-arrays (multiclass
TreeExplainer)
    if hasattr(shap_values, "values"):
        sv = shap_values.values
    else:
        sv = shap_values

    if isinstance(sv, list):
        sv = np.stack(sv, axis=-1)

    if len(sv.shape) == 3:
        # Multiclass: (n_samples, n_features, n_classes)
        # Take mean across classes for each sample
        contributions = np.abs(sv[:, feature_idx, :]).mean(axis=1)
    else:
        # Binary: (n_samples, n_features)
        contributions = sv[:, feature_idx]

    return {
        'feature': feature_name,
        'mean_contribution': np.mean(contributions),
        'median_contribution': np.median(contributions),
        'std_contribution': np.std(contributions),
        'max_positive': np.max(contributions),
        'max_negative': np.min(contributions),
        'positive_count': np.sum(contributions > 0),
        'negative_count': np.sum(contributions < 0)
    }
```

7. ***Main_file***

```
from src.data_processing import load_and_clean
from src.data_processing import add_features
```

```python
from src.data_processing import split_data
from src.model_training import train_and_save
from src.inference import predict_patient

import os
import pandas as pd

MODE = "train"   # change to "train", "inference", or "custom_patient"

DATA_PATH = "data/oasis_longitudinal_demographics.xlsx"


df = load_and_clean(DATA_PATH)
df = add_features(df)
X,y,X_train, X_test, y_train,y_test = split_data(df)

if MODE == "train":
    print("Training models...")
    train_and_save(X, X_train, X_test, y_train, y_test)
    print("\nModels trained and saved in /models folder")


elif MODE == "inference":
    print("Running inference...\n")

    # SINGLE prediction
    sample = X.iloc[[3]]
    result = predict_patient(sample)

    print(f"Prediction: {result['prediction']}")
    print(f"Confidence: {result['confidence']}%\n")

    print("Top Contributing Factors:")
    for signal in result['top_signals']:
        print(f"  • {signal}")

    print("\nOther Contributing Factors:")
    for factor in result['other_factors']:
        print(f"  • {factor}")


elif MODE == "custom_patient":
    print("Running inference on custom patient data...\n")

    patient_data = {
        'Visit': 1,                  # Visit number
        'MR Delay': 0,               # Days since last visit (0 for first
visit)
        'M/F': 1,                    # 1=Male, 0=Female
```

```
        'Age': 75,                    # Patient age in years
        'EDUC': 18,                   # Years of education
        'SES': 2,                     # Socioeconomic status (1-5)
        'MMSE': 28,                   # Mini-mental state exam score (0-30)
        'eTIV': 1506.0,               # Estimated total intracranial volume
        'nWBV': 0.709,                # Normalized whole brain volume
        'ASF': 1.207,                 # Atlas scaling factor
    }


    custom_sample = pd.DataFrame([patient_data])

    custom_sample['Brain_Ratio'] = custom_sample['nWBV'] /
custom_sample['eTIV']
    custom_sample['Age_nWBV'] = custom_sample['Age'] * custom_sample['nWBV']
    custom_sample['nWBV_diff'] = 0  # Longitudinal features (0 for single
sample)
    custom_sample['MMSE_diff'] = 0

    custom_sample = custom_sample[X.columns]

    result = predict_patient(custom_sample)

    print(f"Patient Profile:")
    print(f"  Age: {patient_data['Age']} years")
    print(f"  Gender: {'Male' if patient_data['M/F'] == 1 else 'Female'}")
    print(f"  MMSE Score: {patient_data['MMSE']}/30")

    print(f"Prediction: {result['prediction']}")
    print(f"Confidence: {result['confidence']}%\n")

    print("Top Contributing Factors:")
    for signal in result['top_signals']:
        print(f"  - {signal}")

    print("\nOther Contributing Factors:")
    for factor in result['other_factors']:
        print(f"  - {factor}")


else:
    print("Invalid MODE. Use 'train', 'inference', or 'custom_patient'")
```

8. ***GUI***

```python
from __future__ import annotations

from pathlib import Path

import pandas as pd
import streamlit as st

from src.data_processing import add_features, load_and_clean, split_data
from src.inference import predict_patient
from src.model_training import train_and_save


PROJECT_ROOT = Path(__file__).parent
DEFAULT_DATA_PATH = PROJECT_ROOT / "data" /
"oasis_longitudinal_demographics.xlsx"


st.set_page_config(
    page_title="Dementia Prediction Pipeline",
    page_icon="",
    layout="wide",
)


@st.cache_data(show_spinner=False)
def load_dataset(data_path: str) -> pd.DataFrame:
    df = load_and_clean(data_path)
    df = add_features(df)
    return df


def _feature_columns_from_models() -> list[str] | None:
    try:
        import joblib

        cols_path = PROJECT_ROOT / "models" / "feature_columns.pkl"
        if cols_path.exists():
            cols = joblib.load(str(cols_path))
            if isinstance(cols, list) and all(isinstance(c, str) for c in
cols):
                return cols
    except Exception:
        return None
    return None


def _render_prediction(result: dict) -> None:
    c1, c2 = st.columns([1, 1])
    with c1:
```

```
        st.metric("Prediction", str(result.get("prediction", "")))
    with c2:
        st.metric("Confidence", f"{result.get('confidence', 0):.3f}%")

    st.subheader("Top contributing factors")
    for s in result.get("top_signals", []):
        st.write(f"- {s}")

    st.subheader("Other contributing factors")
    for s in result.get("other_factors", []):
        st.write(f"- {s}")


st.title("Dementia Prediction Model — Training & Inference UI")
st.caption("Run training, sample inference, or custom patient predictions
without editing `main.py`.")

with st.sidebar:
    st.subheader("Dataset")
    data_path = st.text_input("Excel path", value=str(DEFAULT_DATA_PATH))
    st.write("If you move the dataset, update this path.")

    st.subheader("Models")
    st.write("Training writes to `models/` and `outputs/`.")


tabs = st.tabs(["Train models", "Inference (sample)", "Custom patient"])


with tabs[0]:
    st.subheader("Train models")
    st.write("This trains the two-stage cascade and saves artifacts into
`models/`.")

    if not Path(data_path).exists():
        st.error(f"Dataset not found at: {data_path}")
    else:
        df = load_dataset(data_path)
        X, y, X_train, X_test, y_train, y_test = split_data(df)

        st.write("Dataset preview")
        st.dataframe(df.head(10), use_container_width=True)

        if st.button("Train and save models", type="primary"):
            with st.spinner("Training models..."):
                train_and_save(X, X_train, X_test, y_train, y_test)
            st.success("Training complete. Models saved to `models/` and plots
to `outputs/`.")
```

```
with tabs[1]:
    st.subheader("Inference on a dataset row")
    st.write("Pick a row index from the dataset and run the model inference
with SHAP explanations.")

    if not Path(data_path).exists():
        st.error(f"Dataset not found at: {data_path}")
    else:
        df = load_dataset(data_path)
        X = df.drop(columns=["Group", "Subject ID"])

        idx = st.slider("Row index", min_value=0, max_value=len(X) - 1,
value=min(3, len(X) - 1))
        st.write("Selected row (features)")
        st.dataframe(X.iloc[[idx]], use_container_width=True)

        if st.button("Run inference", type="primary"):
            with st.spinner("Predicting..."):
                result = predict_patient(X.iloc[[idx]])
            _render_prediction(result)


with tabs[2]:
    st.subheader("Custom patient")
    st.write("Enter patient measurements. Derived features are computed
automatically.")

    cols = _feature_columns_from_models()
    if cols is None:
        st.info("Could not read `models/feature_columns.pkl` yet. Train models
first for best compatibility.")

    with st.form("custom_patient_form"):
        c1, c2, c3 = st.columns(3)
        with c1:
            visit = st.number_input("Visit", min_value=1, value=1, step=1)
            mr_delay = st.number_input("MR Delay", min_value=0, value=0,
step=1)
            gender = st.selectbox("Gender", options=["Male", "Female"],
index=0)
            age = st.number_input("Age", min_value=18, max_value=120,
value=75, step=1)
        with c2:
            educ = st.number_input("EDUC (years)", min_value=0, max_value=40,
value=18, step=1)
            ses = st.number_input("SES (1-5)", min_value=1, max_value=5,
value=2, step=1)
            mmse = st.number_input("MMSE (0-30)", min_value=0, max_value=30,
value=28, step=1)
```

```
            etiv = st.number_input("eTIV", min_value=0.0, value=1506.0,
step=1.0)
        with c3:
            nwbv = st.number_input("nWBV", min_value=0.0, value=0.709,
step=0.001, format="%.3f")
            asf = st.number_input("ASF", min_value=0.0, value=1.207,
step=0.001, format="%.3f")

        submitted = st.form_submit_button("Predict", type="primary")

    if submitted:
        patient_data = {
            "Visit": int(visit),
            "MR Delay": int(mr_delay),
            "M/F": 1 if gender == "Male" else 0,
            "Age": float(age),
            "EDUC": float(educ),
            "SES": float(ses),
            "MMSE": float(mmse),
            "eTIV": float(etiv),
            "nWBV": float(nwbv),
            "ASF": float(asf),
        }

        custom_sample = pd.DataFrame([patient_data])
        custom_sample["Brain_Ratio"] = custom_sample["nWBV"] /
custom_sample["eTIV"]
        custom_sample["Age_nWBV"] = custom_sample["Age"] *
custom_sample["nWBV"]
        custom_sample["nWBV_diff"] = 0.0
        custom_sample["MMSE_diff"] = 0.0

        if cols is not None:
            missing = [c for c in cols if c not in custom_sample.columns]
            if missing:
                st.error(f"Custom patient is missing required features:
{missing}")
            else:
                custom_sample = custom_sample[cols]
                with st.spinner("Predicting..."):
                    result = predict_patient(custom_sample)
                _render_prediction(result)
        else:
            with st.spinner("Predicting..."):
                result = predict_patient(custom_sample)
            _render_prediction(result)
```

9. ***Model_comparision***

```python
#!/usr/bin/env python

import sys
import os
from pathlib import Path

# Add parent directory to path for imports
sys.path.insert(0, str(Path(__file__).parent))

import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.impute import KNNImputer
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import (confusion_matrix, recall_score, precision_score,
                             f1_score, accuracy_score, classification_report)
from sklearn.ensemble import RandomForestClassifier,
GradientBoostingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from xgboost import XGBClassifier
import matplotlib.pyplot as plt
import seaborn as sns

from data_processing import load_and_clean, add_features


def train_and_evaluate_models(X_train, X_test, y_train, y_test):
    """Train multiple models and return metrics."""

    models = {
        'XGBoost': XGBClassifier(
            n_estimators=300, max_depth=4, learning_rate=0.05,
            random_state=42, eval_metric='mlogloss', verbose=0
        ),
        'Random Forest': RandomForestClassifier(
            n_estimators=200, max_depth=10, random_state=42, n_jobs=-1
        ),
        'Gradient Boosting': GradientBoostingClassifier(
            n_estimators=200, max_depth=5, learning_rate=0.1, random_state=42
        ),
        'Logistic Regression': LogisticRegression(
            max_iter=1000, random_state=42, n_jobs=-1
        ),
```

```python
        'SVM (RBF)': SVC(kernel='rbf', random_state=42, probability=True),
        'KNN (k=5)': KNeighborsClassifier(n_neighbors=5, n_jobs=-1),
        'KNN (k=10)': KNeighborsClassifier(n_neighbors=10, n_jobs=-1),
    }

    results = {}

    for name, model in models.items():
        print(f" Training {name}...")

        try:
            # Train
            model.fit(X_train, y_train)
            y_pred = model.predict(X_test)

            # Calculate metrics
            cm = confusion_matrix(y_test, y_pred)
            recall = recall_score(y_test, y_pred, average='weighted',
zero_division=0)
            precision = precision_score(y_test, y_pred, average='weighted',
zero_division=0)
            f1 = f1_score(y_test, y_pred, average='weighted', zero_division=0)
            accuracy = accuracy_score(y_test, y_pred)

            results[name] = {
                'model': model,
                'predictions': y_pred,
                'confusion_matrix': cm,
                'recall': recall,
                'precision': precision,
                'f1': f1,
                'accuracy': accuracy
            }

            print(f"{name} trained successfully")
            print(f"\n Metrics for {name}:")
            print(f"   Accuracy:  {accuracy:.4f}")
            print(f"   Recall:    {recall:.4f}")
            print(f"   Precision: {precision:.4f}")
            print(f"   F1-Score:  {f1:.4f}")

        except Exception as e:
            print(f"Error training {name}: {str(e)}")

    return results


def plot_confusion_matrices(results, y_test_labels):
    """Plot confusion matrices for all models."""
```

```
    n_models = len(results)
    fig, axes = plt.subplots(2, 4, figsize=(18, 10))
    axes = axes.flatten()

    for idx, (name, data) in enumerate(results.items()):
        cm = data['confusion_matrix']

        sns.heatmap(
            cm, annot=True, fmt='d', cmap='Blues', ax=axes[idx],
            xticklabels=y_test_labels, yticklabels=y_test_labels
        )
        axes[idx].set_title(f"{name}\nRecall: {data['recall']:.3f} | Acc:
{data['accuracy']:.3f}")
        axes[idx].set_xlabel('Predicted')
        axes[idx].set_ylabel('Actual')

    # Hide extra subplots
    for idx in range(n_models, len(axes)):
        axes[idx].set_visible(False)

    plt.tight_layout()
    plt.savefig('outputs/model_comparison_confusion_matrices.png', dpi=300,
bbox_inches='tight')
    print("\nConfusion matrices saved to
outputs/model_comparison_confusion_matrices.png")
    plt.show()


def plot_metrics_comparison(results):
    """Plot comparison of key metrics across models."""
    metrics_data = {
        'Model': list(results.keys()),
        'Accuracy': [data['accuracy'] for data in results.values()],
        'Recall': [data['recall'] for data in results.values()],
        'Precision': [data['precision'] for data in results.values()],
        'F1-Score': [data['f1'] for data in results.values()]
    }

    metrics_df = pd.DataFrame(metrics_data)

    # Plot
    fig, axes = plt.subplots(2, 2, figsize=(14, 10))

    metrics = ['Accuracy', 'Recall', 'Precision', 'F1-Score']
    for idx, (ax, metric) in enumerate(zip(axes.flatten(), metrics)):
        ax.barh(metrics_df['Model'], metrics_df[metric], color='steelblue')
        ax.set_xlabel(metric)
        ax.set_xlim([0, 1])
```

```python
        # Add value labels
        for i, v in enumerate(metrics_df[metric]):
            ax.text(v + 0.02, i, f'{v:.3f}', va='center')

        ax.set_title(f'{metric} Comparison')

    plt.tight_layout()
    plt.savefig('outputs/model_comparison_metrics.png', dpi=300,
bbox_inches='tight')
    print("Metrics comparison saved to outputs/model_comparison_metrics.png")
    plt.show()

    return metrics_df


def print_summary_table(results):
    """Print summary table of all models."""
    summary_data = []

    for name, data in results.items():
        summary_data.append({
            'Model': name,
            'Accuracy': f"{data['accuracy']:.4f}",
            'Recall': f"{data['recall']:.4f}",
            'Precision': f"{data['precision']:.4f}",
            'F1-Score': f"{data['f1']:.4f}"
        })

    summary_df = pd.DataFrame(summary_data)

    print("\n" + "="*80)
    print("[*] MODEL COMPARISON SUMMARY")
    print("="*80)
    print(summary_df.to_string(index=False))
    print("="*80)

    return summary_df


def main():
    print("\n" + "="*80)
    print("[ML] MODEL COMPARISON SUITE")
    print("="*80)

    # Load and prepare data
    print("\n[*] Loading data...")
    df = load_and_clean("data/oasis_longitudinal_demographics.xlsx")
    df = add_features(df)
```

```python
X = df.drop(columns=["Group", "Subject ID"])
y = df["Group"]

print(f"   Data shape: {X.shape}")
print(f"   Classes: {y.unique()}")

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Preprocess
print("\n[*] Preprocessing...")
imputer = KNNImputer(n_neighbors=5)
scaler = StandardScaler()

X_train = imputer.fit_transform(X_train)
X_train = scaler.fit_transform(X_train)
X_train = pd.DataFrame(X_train, columns=X.columns)

X_test = imputer.transform(X_test)
X_test = scaler.transform(X_test)
X_test = pd.DataFrame(X_test, columns=X.columns)

# Encode labels
le = LabelEncoder()
y_train_encoded = le.fit_transform(y_train)
y_test_encoded = le.transform(y_test)

print(f"   Training set: {X_train.shape}")
print(f"   Test set: {X_test.shape}")

# Train and evaluate models
print("\n" + "="*80)
print("[*] TRAINING MODELS")
print("="*80)

results = train_and_evaluate_models(
    X_train, X_test, y_train_encoded, y_test_encoded
)

# Generate reports
print("\n" + "="*80)
print("[*] GENERATING REPORTS")
print("="*80)

# Summary table
summary_df = print_summary_table(results)
```

```python
    # Save summary to CSV
    summary_df.to_csv('outputs/model_comparison_summary.csv', index=False)
    print("\n[OK] Summary saved to outputs/model_comparison_summary.csv")

    # Confusion matrices
    print("\n[*] Plotting confusion matrices...")
    plot_confusion_matrices(results, le.classes_)

    # Metrics comparison
    print("\n[*] Plotting metrics comparison...")
    plot_metrics_comparison(results)

    # Detailed classification reports
    print("\n" + "="*80)
    print("[*] DETAILED CLASSIFICATION REPORTS")
    print("="*80)

    for name, data in results.items():
        print(f"\n{name}:")
        print(classification_report(y_test_encoded, data['predictions'],
                                    target_names=le.classes_))

    print("\n" + "="*80)
    print("[OK] MODEL COMPARISON COMPLETE")
    print("="*80)
    print("\n[Output] Output files saved to outputs/:")
    print("   - model_comparison_summary.csv")
    print("   - model_comparison_confusion_matrices.png")
    print("   - model_comparison_metrics.png")
    print("\n[!] Key Findings:")

    best_model = max(results.items(), key=lambda x: x[1]['f1'])[0]
    best_recall = max(results.items(), key=lambda x: x[1]['recall'])[0]

    print(f"   - Best F1-Score: {best_model}")
    print(f"   - Best Recall: {best_recall}")
    print(f"   • XGBoost F1-Score: {results['XGBoost']['f1']:.4f}")
    print(f"   • XGBoost Recall: {results['XGBoost']['recall']:.4f}")


if __name__ == "__main__":
    main()
```